

k reference architecture. If you need to set up something like a 25k reference architecture, you may need to change things that are not covered in this guide. Alternate reference architectures can be found [here](#).

One time steps:

- Clone the GET repo and cd into it
- Copy bootstrap.sh from this MR to the root and update it as necessary
- You may need to make it executable: `chmod +x bootstrap.sh`

Note, bootstrap.sh has steps that only need to be ran once, as well as steps that need to be ran for setting up a new \$GCPENVPREFIX, and they still need to be separated.

Steps to add a new \$GCPENVPREFIX:

- Use Provisioning the environment with Terraform as guide for setting up Terraform, ignoring the AWS steps as we are using GCP

- Make sure you are within your cloned GET repo
- Update the variables in bootstrap.sh as necessary
- Run `./bootstrap.sh`

- Note the ip address at the end

- Note the username created when the ssh key is added to the service account (will be used later)

- Run `mkdir -p terraform/environments/$GCPENVPREFIX`

- Copy over environment.tf, main.tf, and variables.tf from this MR into the corresponding directory

- Update those .tf files as necessary

- Add the ip address you obtained earlier in variables.tf under externalip

- Add a proper service account prefix in environment.tf

- Update prefix in main.tf to match the value of \$GCPENVPREFIX

- Cd to `/terraform/environments/$GCPENVPREFIX`

- Run `terraform init`

- Run `terraform apply`

- Small celebration :tada:

- Use Configuring the environment with Ansible as guide for setting up Ansible, ignoring the AWS steps as we are using GCP

- Cd to the root directory of the GET repo

- Run `mkdir -p ansible/environments/$GCPENVPREFIX/files/gitlabconfigs`

- Run `mkdir -p ansible/environments/$GCPENVPREFIX/files/gitlabtasks`

- Run `mkdir -p ansible/environments/$GCPENVPREFIX/inventory`

- From the MR we referenced earlier, copy `environments/gcp-2k/files/gitlabconfigs/gitlabrails.rb.j2` to `ansible/environments/$GCPENVPREFIX/files/gitlabconfigs` · this is necessary for the license to be added correctly when the configuration is applied

- Copy over vars.yml, and gcp2k.gcp.yml from the MR into that /inventory directory

- Rename gcp2k.gcp.yml to match \$GCPENVPREFIX (but use underscores) and update both .yml files as necessary

- Copy over monitor.yml to `ansible/environments/$GCPENVPREFIX/files/gitlabtasks`

- If you would like to setup monitoring, you may need to copy other files and folders as well:

Copy over dashboards.yml to ansible/environments
Copy over datasources.yml to ansible/environments
Copy over linux-package folder to ansible/environments
Nothing needs to change in monitor.yml, but be sure grafanapassword is set in vars.yml
Add the username obtained earlier (when running .bootstrap.sh) to the vars.yml file as ansibleuser
Make sure to configure other variables like passwords/secrets when necessary (depends on the reference architecture but at least the gitlabrootpassword, postgrespassword, gitaltoken and redispassword will be required)
You will likely also have to update the prefix, externalurl and uncomment a few lines like gitlablicensefile
Consult the documentation and other reference architecture files (e.g. the one for 25k ref architecture) for other variables you may need to update
Acquire a new Ultimate license using the Support Super Form
Add the license file (without renaming it) to /ansible/environments/\$GCPENVPREFIX/files
From the root directory, follow the steps in Installing Ansible with a Virtual Environment
Cd to the ansible directory
Run ansible-playbook -i environments/\$GCPENVPREFIX/inventory playbooks/all.yml
After logging in to the instance, if the Ultimate license doesn't apply, you may have to manually upload the license

Setting up an existing environment (\$GCPENVPREFIX)

Clone the GET repo, if it doesn't already exist, and cd into it
Run mkdir terraform/environments/\$GCPENVPREFIX
Run mkdir -p ansible/environments/\$GCPENVPREFIX/files/gitlabtasks
Navigate to <https://gitlab.com/gitlab-org/secure/pocs/gitlab-environment-toolkit-configs>
(pending merge of this MR)
Copy the configs/\$GCPENVPREFIX/terraform/.tf files into the terraform/environments/\$GCPENVPREFIX directory
Copy the configs/\$GCPENVPREFIX/ansible/.yml files into the ansible/environments/\$GCPENVPREFIX directory
From the root directory, follow the steps in Installing Ansible with a Virtual Environment
Cd to the ansible directory
Run ansible-playbook -i environments/\$GCPENVPREFIX/inventory playbooks/all.yml

Populating data

Clone the GPT repo
Split the importing of data into the following two steps

Horizontal data:

console

docker run -it \

-e ACCESSTOKEN=glpat-REDACTED \

-e GPTGENERATORPOOLTIMEOUT=600 -e GPTGENERATORPOOLSIZE=1 -e GPTGENERATORRET
GPTGENERATORRETRYWAIT=10 \

-v \$PWD:/results \

-v \$PWD:/config \

```
gitlab/gpt-data-generator \  
--environment gcp-2k.json --environment-url=https://34.83.26.81 \  
--subgroups 10 --projects 10 --no-vertical
```

This command will most likely time out, but if it shows as importing in the browser, just wait for it to finish importing.

Vertical data:

```
console  
docker run -it \  
-e ACCESSTOKEN=glpat-REDACTED \  
-e GPTDEBUG=true \  
-e GPTGENERATORPOOLTIMEOUT=600 -e GPTGENERATORPOOLSIZE=1 -e GPTGENERATORRETRYWAIT=10 \  
gitlab/gpt-data-generator --environment gcp-2k.json --no-horizontal
```

Re-deploying a new build

There are prerequisite steps that will be covered in the Setting up a GET section in the future

Navigate to your cloned GET repo and cd into it

If not already activated for your terminal, run `./get-python-env/bin/activate`

cd into the ansible directory of your cloned GET repo

Check `ansible/environments/$GCPENVPREFIX/inventory/vars.yml` to see which of the 3 methods of deployment we are

targeting, and if anything needs to be changed before deploying

Redeploy with the same ansible command from the setup steps: `ansible-playbook -i environments/$GCPENVPREFIX/inventory playbooks/all.yml`

Monitoring

The Grafana dashboards can be found at `/-/grafana/dashboards/`. Login credentials can be found in 1password under "Static Analysis - 2K GCP Grafana".

Setting up the environment is covered in previous sections, but the files necessary to make this work are as follows:

```
ansible/environments/gcp-2k/files/gitlabtasks/monitor.yml  
ansible/environments/linuxpackage/server-performance.json  
ansible/environments/dashboards.yaml  
ansible/environments/datasources.yaml
```

These files can be found in this MR.

See the Custom Tasks section of the GET docs for more info.

Resources/Acronyms

GitLab Environment Toolkit (GET)

GitLab Performance Tool (GPT)

The Static Analysis GCP Project: dev-sast-prereceive-8a4574ec

GCP - Google Cloud Platform

Secret Push Protection documentation

Miscellaneous

Most examples throughout the runbook have been added using the gcp-2k environment as their basis.

Rails console and checking feature flags

SSH in to a rails server: `gcloud compute ssh --zone us-west1-c gcp-2k-gitlab-rails-1 --project dev-sast-prereceive-8a4574ec`

Launch a rails console: `sudo gitlab-rails console` (it will take a bit to connect)

Check the status of the push check feature for a project:

`Feature.enabled?(:prereivesecretdetectionpushcheck, Project.find(123))`

Enabling the Application Setting

Via the web, enable the feature instance-wide by navigating to `/admin/applicationsettings/securityandcompliance`

Via the rails console:

```
ruby
a = ApplicationSetting.first
a.prereivesecretdetectionenabled = true
a.save!
```

Determining current version

To see the current version of GitLab running on the instance, login and then go to `/help`.

SECTION: engineering/development/sec/secure/secret-detection/runbooks/secret-push-protection-troubleshooting.md

When to use this runbook?

Use this runbook for troubleshooting Production issues related to the

secret push protection feature.

Relevant settings

Setting	Type	Level	Visibility
prereceivesecretdetectionbetarelease	Feature Flag	Instance	Not visible, has to be toggled via ChatOps.
prereceivesecretdetectionenabled	Database Setting	Instance	Only in a Dedicated instance, or with prereceivesecretdetectionbetarelease enabled and only when the feature is licensed (in Ultimate).
prereceivesecretdetectionpushcheck	Feature Flag	Project	Not visible, has to be toggled via ChatOps.
prereceivesecretdetectionenabled	Database Setting	Project	Only in a Dedicated instance, or with prereceivesecretdetectionenabled enabled and only when the feature is licensed (in Ultimate).

Monitoring

Secret push protection monitoring is the preferred dashboard for monitoring the feature to help identify and mitigate any reliability issues or performance regressions that may occur when it is enabled on GitLab.com.

Gitaly Latency Dashboard for the PreReceiveHook method may be used as a supplement.

Enabling/Disabling

Enabling/Disabling for GitLab.com

To disable the feature across all of GitLab.com, there are two options.

The first option, and most likely quickest, is to disable the feature via ChatOps. Use the following command: `/chatops run feature set prereceivesecretdetectionpushcheck false`

The second option is to disable pre-receive secret detection, as described in the docs. This will require a change access request (production example), as it requires administrator access.

Either of these options will disable the feature for all GitLab.com users and should be used in cases where the feature is causing significant performance degradation.

Other properties that can be enabled/disabled

Enabling/Disabling for a project

This can only be changed by a Maintainer, or above, for the project and is the preferred method for disabling a project.

With the `prereceivesecretdetectionpushcheck` feature flag on for a given project, secret push protection can be toggled for the

project through the security configuration page for the project. Instructions can be found in this in-progress documentation MR, but will eventually be added to the docs.

When both the feature flag and the setting are on, git pushes to the project should be stopped when they contain a secret. As a test, glpat-12345678901234567890 should be prevented when pushing (from the terminal, the Web IDE, etc.)

The prereceivesecretdetectionpushcheck feature flag

The prereceivesecretdetectionpushcheck feature flag will be used to enable/disable secret push protection for a given project.

It will also be used in conjunction with the secret push protection setting in a project's Security Configuration. Both the feature flag and the setting will need to be on for secret push protection to work for a given project.

Check status:

```
/chatops run feature get prereceivesecretdetectionpushcheck
```

Enable for a project:

```
`/chatops run feature set --project=the-namespace/of-the-project  
prereceivesecretdetectionpushcheck true`
```

Disable for a project:

```
`/chatops run feature set --project=the-namespace/of-the-project  
prereceivesecretdetectionpushcheck false`
```

The prereceivesecretdetectionbetarelease flag

The prereceivesecretdetectionbetarelease flag will be used to enable the Secret Detection section in the Security and Compliance Admin section (<<https://gitlab.com/admin>> -> Settings -> Security and Compliance), but turning the flag on does not turn on pre-receive secret detection. Conversely, turning the flag off does not turn the feature off.

Check the status of the prereceivesecretdetectionbetarelease feature flag:

```
/chatops run feature get prereceivesecretdetectionbetarelease
```

Enable the feature flag:

```
/chatops run feature set prereceivesecretdetectionbetarelease true
```

Disable the feature flag:

```
`/chatops run feature set prereceivesecretdetectionbetarelease  
false`
```

Resolving and skipping blocked pushes

Resolving a blocked push and Skipping secret detection are both documented in the in-progress documentation MR.

Viewing logs

Secret Detection audit log events are documented here.

For GitLab.com, all logged events can be viewed here[&a=h@bd3f1e1](#)), and logs of just blocked push events due to detected secrets can be viewed here.

SECTION: engineering/development/sec/secure/static-analysis/_index.md

Static Analysis

The Static Analysis group at GitLab is charged with developing the Static Application Security Testing (SAST) feature category for customer software repositories.

Links

- Slack channel: [gast-static-analysis](#)
- Team members: [Static Analysis group](#)

How We Work

The Static Analysis group is largely aligned with GitLab's Product Development Flow, however there are some notable differences in how we seek to deliver software. The engineering team predominantly concerns itself with the delivery of software, which is the portion of the workflow states where we deviate the most. What follows is how we manage the handoff from product management to engineering to deliver software.

Issues worked by this team can span analyzers, vendored templates, CI/CD components, and GitLab's Rails monolith.

Planning

The Static Analysis group works on a monthly planning cadence, much like all groups across the company. We are product-driven and work in response to the priorities identified by Product Management.

As a guideline, the team capacity is allocated to:

- 60% Features (Product priorities)
- 40% Maintenance and bugs (Engineering priorities)

We use planning issues to determine what our team is going to be working on during each milestone. Planning issues are the single-source-of-truth from which engineers do work. If an issue is not in the planning issue, it's unlikely that we're working on it.

Planning issues for 17.x.

This link is also available on our Slack channel bookmarks.

How we interact with planning issues

- Engineering Manager adds to the planning issue:
 1. Feature epics and standalone issues based on Product priorities.
 2. Maintenance epics, standalone issues, and bugs based on Engineering priorities.
 3. The engineering allocation for the milestone; i.e.: DRIs for epics, feature vs maintenance allocation, and reaction rotation.
- Engineering Manager mentions engineers requesting that they:
 1. Review their allocation.
 2. Commit to deliverables by setting the ~Deliverable label to chosen issues.
 3. Update labels, milestone and health status on selected issues.
- Engineering Manager mentions Product manager for review.

Static Analysis Shared Calendar

The Static Analysis Shared Calendar is used to make sure PTO events are visible to everyone on the team.

Below are the steps to add the calendar to Time Off by Deel:

- In Slack, jump to Time Off by Deel > Home, click on the dropdown Your Events, and select Calendar Sync.
- Under Additional calendars to include?, click on Add calendar.
- Add the following calendar ID:
cfb285ec72974733f23fd84f70397732e68f7db9abe706c5613f199b6202e379a@group.calendar.google.com
- Great job! · Your PTO events will be synced to Static Analysis Shared Calendar from now on. ·

Observability

For GitLab.com, we monitor performance of our code within the Rails application, metrics around our CI build performance, and traffic to our container registries. These dashboards are accessible on the Monitoring page.

Observability is a critical component to any high-availability system and it is recommended for each team member to review each dashboard and ensure they are familiar with their usability and trends.

- Static Analysis Group Dashboard

- SAST Analyzer Registry

Traffic%2CrefreshInterval%3A(pause%3A!t%2Cvalue%3A0)%2Ctime%3A(from%3Anow-24h%2Cto%3Anow

- SAST Engineering Kibana Dashboard

Runbooks

The process for monitoring, responding to, and mitigating incidents is documented within our Static Analysis Runbooks page.

Software delivery in Static Analysis

We follow the AST stage Planning process.

How we commit to delivering work in a milestone

In GitLab, the ~Deliverable label is referred to as a release scoping label. Applying this label

represents a commitment from the engineering team to realize the work required in the issue within the milestone to which the issue is assigned. This means we decide whether we can commit to delivering work once an issue is in the workflow::ready for development state.

The decision on when to use the ~Deliverable label is made through answering the following questions.

- Given the issue's weight, are we reasonably confident there is enough time left in the milestone for the engineer to deliver the issue?

- We currently assume that an issue with weight 9 can be delivered in a single milestone.

- Would the issue be achievable early in the next milestone if work began now?

- If so, discuss with the Product Manager about the situation. Work can begin if the Product Manager agrees with the proposed timeline and would like to proceed.

- Please make sure the milestone is updated before continuing with work.

- Is this the smallest, testable unit of work which adds value and cannot be further broken down without adding overhead?

The ~Deliverable label is applied if the answer to the above questions are yes. The use of this label impacts the group's Say/Do ratio, making the Engineering Manager the directly responsible individual for this label. However, engineers in Static Analysis are empowered to use their judgment about applying this label and proceeding if they believe the work is achievable.

Please

have a conversation with the Engineering Manager if uncertain about how to proceed.

Code Review Process

The process for reviewing and maintaining code is documented within our Static Analysis Group Code Review page.

Security Vulnerability Process

We are responsible to ensure that what we deliver is secure. This means that we dogfood GitLab's Security features.

See the vulnerability management process.

When creating an issue for a vulnerability, please make sure to follow the Engineering Security instructions.

SLO by Vulnerability Severity

When triaging Unknown vulnerabilities, they should be assigned a proper severity as a means to decide the priority they should receive to be resolved. The corresponding priority is taken from issue triage.

Target	Unknown	Critical	High	Medium	Low
-----	-----	-----	-----	-----	-----
--					
Dismiss/Confirm Vuln	72h	72h	72h	1mo	1mo
Confirmed Vuln is Resolved	N/A	~priority::1	~priority::2	~priority::3	~priority::4

Workstream Designation

The following is a description of the type of work and which workstream it flows through.

Work	Responsible Workstream
---	---
Triage of new vulns	This should be done as a part of the MR review that introduces the vulns.
Triage of existing vulns	This is done by the main maintainer of each of our analyzers as defined in our Release project's issue template.
Resolution of Critical / High Vulns	These should be a Product-driven priority.
Resolution of Medium / Low Vulns	This is done by the main maintainer of each of our analyzers as defined in our Release project's issue template.

As always, contributions are welcome from our community or the current MR coach in rotation.

False Positive Dismissal Process

The process for dismissing a vulnerability as a false positive is as follows:

- If it doesn't exist on the Static Analysis Group Defined False Positives page, then write documentation describing the type of false positive and why we think it is classified as such.
- If the vulnerability relates to a specific code location (e.g. SAST), then open an MR with comments at each FP location that contain a link to the FP documentation.
- Dismiss vulnerability in the GitLab UI with a comment that contains:

- A link to the FP documentation.
- A link to the FP comment MR if it was created.

Vulnerability Issue Labels

When creating issues for vulnerability consider adding the following labels besides our normal labels:

- ~security
- ~"type::bug"

When there is a doubt about the severity/priority while creating the issue and severity/priority labels are

not added. Then Appsec Escalation Engine

could be leveraged to initiate a discussion with the Appsec team. This bot monitor issues that are labeled

~security and not ~test or ~"type::feature". If severity/priority labels are not present, then labels

security-sp-label-missing and security-triage-appsec will be added and this issue will be mentioned in the

\sec-appsec Slack channel. Then, the appsec stable counterpart for the group or App sec team triage person

will pick up the issue and assign a severity as part of the appsec triage rotation.

We Own What We Ship

The security analyzers we develop may rely heavily upon open source software.

This means we can be dramatically affected by changes in those software packages. We will check for updates to these packages once per GitLab release. New versions will be scrutinized for the following aspects:

- Breaking changes
- New, updated, or removed security rules
- Behavior changes
- Analyzer changes needed to use the new version
- Security vulnerabilities

An issue will be created and prioritized if a breaking change is discovered. Otherwise, dependency updates will be detailed in the relevant

analyzer's changelog and a new version will be released utilizing the change. This is a lot of work, most likely requiring several hours of

focused study to understand what is happening in the new version. As a result, dependency updates will be divided evenly and assigned to

Senior and Intermediate Backend Engineers, with the remainder going to the group's Staff Backend Engineer. Assignments will be managed through our Release project's issue template.

The assigned backend engineer is the group's primary liaison with the dependency's open source community. Engineers are expected to contribute back to those projects, especially if critical or high security findings are confirmed.

Testing for security vulnerabilities

We have a dependencies group which contains mirrored copies of the OSS projects upon which we most rely. Prior to submitting an MR updating an analyzer to a new version of these projects, engineers are expected to do the following:

1. Find a release branch which matches the new version we wish to ship.
1. If one doesn't exist, create it from the corresponding tag.
1. Push the branch through a pipeline which executes all of our security products.
1. Please note, some of these projects have complicated builds. Auto DevOps works sometimes, but projects such as spotbugs can require a custom CI configuration for our scans to be successful. Also, these projects include tests that can be noisy if not filtered out.
1. Evaluate any potential security vulnerabilities which are found.
1. Work with the relevant Open Source community to resolve any Critical or High severity findings.
1. GitLab has published Secure Coding Guidelines, which may be a useful resource to use when trying to solve identified risks.

We do not want to ship updated dependencies which have Critical and High severity vulnerabilities in them. If we find ourselves in this situation, we will withhold updates to the dependency until the problems have been patched.

Go security fixes

At times we will need to update our analyzers because of security updates to go lang itself. In this situation, we follow the established release process.

Testing and validation

Our users expect us to provide them with a quality experience, no matter which open source or proprietary components we include in our analyzers.

They also expect our documentation to clearly outline the configurations we support so that they can make informed decisions about whether to adopt our tools for their needs.

Before we document that we support a configuration, we do validate that it works. For example, before we list a certain type of file or build configuration as a supported feature, we must have checked it at least once, however minimally.

However, we do not independently reproduce all end-to-end tests for components we rely on. Maintaining these tests independently would require unnecessary effort and would duplicate work that would be better contributed upstream if it's lacking. Instead, we aim to build tests that cover basic configurations for smoke-testing and demonstration purposes.

We may choose to document supported configurations once they're validated, even if the test coverage is not yet complete.

Unplanned work

In general, the Static Analysis group has two sources of unplanned work: community

contributions and ~priority::1 bugs. We will reserve capacity each release so we can respond quickly and efficiently. In both scenarios, we will route community contributions to the engineer who "owns" the analyzer.

We do, however, own and contribute to projects beyond the analyzers shipped as part of GitLab's product. Where possible, unplanned work requiring the attention of an engineer in Static Analysis will be routed according to that project's CODEOWNERS file. Otherwise, unplanned work will be considered and handled on a case-by-base basis.

Support to customers and prospects

While we plan our work on a monthly basis, customers and customer-facing team members may need support on an unplanned basis.

We aim to support these requests quickly because they affect the success of our customers and our business.

Generally, we aim to provide an initial response and triage the question/report as quickly as is reasonable.

"Reasonable" means, for example, that team members are answering during their normal working hours and are continuing their normal work activities.

Whoever is available and can contribute to a solution is encouraged to make first contact with the questioner and ask any clarifying questions. remember, you can always tag in another group member later if you're unable to resolve the question.

The aim of the triage is to support other team members in moving forward; if development work is required to address the problem, it is not automatically a top priority for the group and should not automatically displace existing planned work.

If there is any question of whether a bug fix or improvement should be taken up immediately, the Engineering Manager and Product Manager should be alerted to facilitate a decision.

When a Customer Success Escalation is declared, the Engineering Manager and Product Manager should both be alerted, and an appropriate team member should be designated to deprioritize existing work and respond to the escalation as soon as possible.

```
{{< tableau height="600px" toolbar="hidden" src="https://10az.online.tableau.com//site/gitlab/v  
iews/TopEngineeringMetrics16989570521080/TopEngineeringMetricsDashboard" >}}  
  {{< tableau/filters "GROUPLABEL"="static analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px"  
src="https://10az.online.tableau.com//site/gitlab/views/MRMetrics/OverallMRsbyType1" >}}  
  {{< tableau/filters "GROUPLABEL"="static analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/DRAFTFlakyte  
stissues/MonthlyFlakyTestIssues" >}}  
  {{< tableau/filters "GROUPNAME"="static analysis" >}}  
{{< /tableau >}}
```

```
{{< tableau height="600px" src="https://10az.online.tableau.com//site/gitlab/views/SlowRSpecTestsIssues/SlowRSpecTestsIssuesDashboard" >}}
  {{< tableau/filters "GROUPLABEL"="static analysis" >}}
{{< /tableau >}}
```

SECTION: engineering/development/sec/secure/static-analysis/code_review.md

Overview

This page documents the code review process utilized by the Static Analysis team.

Reviewer

All Static Analysis group members are expected to regularly participate in code reviews. This includes contributions from both GitLab Team Members and the wider community, as done as a part of our MR coach rotation.

While all team members are encouraged to review merge requests, the ability to merge MRs and release changes is restricted to project maintainers.

Maintainer

To quote from the Engineering Review Workflow:

> Great engineers are often also great reviewers, but code review is a skill in and of itself and not every engineer, no matter their seniority, will have had the same opportunities to hone that skill. It's also important to note that a big part of being a good maintainer comes from knowing the existing product and codebase extremely well, which lets them spot inconsistencies, edge cases, or non-obvious interactions with other features that would otherwise be missed easily.

>

> To protect and ensure the quality of the codebase and the product as a whole, people become maintainers only once they have convincingly demonstrated that their reviewing skills are at a comparable level to those of existing maintainers.

As with regular reviewers, maintainers can be found on the team page, or on the list of GitLab Engineering Projects.

Path to maintainership

Becoming a reviewer

All SAST BE Engineering team members are expected to review changes to the SAST analyzers. As part of the final steps of the onboarding process, the team member page should be updated to add the team member as a reviewer for each SAST analyzer.

yaml

```
secure-SAST-kics: reviewer
```

secure-SAST-kubesec: reviewer
secure-SAST-pmd-apex: reviewer
secure-SAST-security-code-scan: reviewer
secure-SAST-semgrep: reviewer
secure-SAST-sobelow: reviewer
secure-SAST-spotbugs: reviewer
secure-SAST-tracking-calculator: reviewer
secure-common-command: reviewer
secure-common-report: reviewer
secure-common-ruleset: reviewer

Path to maintainership

All senior (and above) Backend engineers who contribute to any of the SAST analyzers are expected to pursue maintainership of all SAST analyzers, but becoming a maintainer is encouraged for all SAST engineers.

All maintainership approvals should be given by at least two existing maintainers.

There are two tiers of maintainership within SAST.

Tier 1

Some analyzers are rarely updated; These include:

kics
kubesec
pmd-apex
security-code-scan
sobelow
spotbugs

Becoming a maintainer on a project that is rarely updated can be tough since there are limited opportunities to contribute or review. In this case, contributions to other GitLab analyzers should be considered and granting of maintainership should be left to the existing maintainer's discretion. The existing maintainer should document the reasons for approval, or if maintainership is denied, suggestions on which projects to contribute to should be given. When reviewing contributions, consideration should be given to contributions to issues within section-sec-request-for-help should also be included, even if these contributions did not result in an MR.

Tier 2

This tier of maintainership includes:

Semgrep
sast-rules
tracking-calculator
ruleset

The granting of maintainership is down to the discretion of the two current maintainers

approving the request, but an applicant must have a track record of contributions to the project. These contributions should demonstrate an understanding of the project and how changes to it could affect the monolith.

Reviewer mentorship program

We follow the same reviewer mentorship program as documented in the Engineering Review Workflow.

Gotchas

There are certain aspects to maintaining our Secure projects that require a critical eye. These are responsibilities that every reviewer within the team should be cognizant of and are expectations on the part of our maintainers. Some examples include:

1. Execution of downstream pipelines prior to merge: this should occur automatically for members of gitlab-org but require manual triggering for community contributions
1. Failing pipelines must be fixed prior to tagging releases, see versioning and release process
1. Analyzers must be kept in sync with our CI templates. In certain cases the analyzer versions must be kept in sync, such as mobsf

SECTION: engineering/development/sec/secure/static-analysis/ec2_dev_environment_setup.md

Static Analysis Group EC2 development machine setup guide

Some of the SAST analyzers, including Semgrep, do not run on the M1 mac as ARM64 is not supported.

To overcome this, a SAST developer image has been added to the EC2 AMI catalog and shared with the `arn:aws:organizations::663740273027:organization/o-qjrbh9ll6x` sandbox organisation so that SAST contributors can run SAST analyzers in a development environment. To setup the image:

1. Create an AWS sandbox account for yourself by visiting <https://gitlabsandbox.cloud/login> and clicking on Create individual Account.
 - For the Cloud Provider, select `aws-51eab1fa` (Master Account 663740273027).
 - For the Organizational Unit, select `eng-dev-sandbox-f1098f70`.
1. When the account has finished being provisioned, click on View IAM Credentials and note the Username and Password.
1. Click on Open AWS Web Console and use the Username and Password from step 2 above.
1. Once logged in to the AWS Web Console, click on the hamburger menu on the left hand side to open up the Console Home sidebar.
1. Click on All services in the sidebar, then navigate to Compute -> EC2.
1. Choose the Sydney region (`ap-southeast-2`) in the region selector at the top right
1. Use the sidebar on the left to navigate to Images -> AMI Catalog.
1. Once the AMI Catalog page has loaded, click on My AMIs.
1. If you see the message No results were found in My AMIs with your current filters., then click on the Clear Filters button.

1. You should now see the static-analysis-workspace image with the description A fast path to Static Analysis analyser development.

Alternatively, you can search for static-analysis-workspace to filter the list of AMIs.

1. Select the static-analysis-workspace AMI and click "Launch Instance with AMI".
1. For the instance type, c6a.xlarge is recommended as it should provide decent single core performance which is useful for the integration tests.
1. In "Network Settings", ensure the check box "Allow SSH traffic from" is selected and choose "My IP" in the dropdown select.
1. Create or select a key pair and download the private key .pem.
1. Click on "Launch Instance". You'll be redirected to another page and see a message stating "Successfully initiated launch of instance".
1. Click on "View all instances" and note the public IP address of the instance.
1. SSH into the instance using `ssh -i <path-to-pem> ubuntu@<ip>`.

Note: If you see a warning message such as WARNING: UNPROTECTED PRIVATE KEY FILE!! along with Permissions 0644 for '<path/to/private-key.pem>' are too open., then you'll need to change the mode of the file using `chmod 600 <path/to/private-key.pem>`

1. You should be greeted by the MOTD.
1. The image has a number of analyzers already cloned, however, these analyzers have been cloned using ssh, so you won't be able to fetch the latest changes for the analyzer unless you copy the SSH key from your local development machine to the EC2 instance.

To avoid copying your SSH key, simply force git to use https instead of ssh:

```
shell
git config --global url."https://gitlab.com/".insteadOf "git@gitlab.com:"
```

SECTION: engineering/development/sec/secure/static-analysis/false_positives.md

Static Analysis Group Defined False Positives

The table below lists different rule ids and situations in which they can be considered as false positives. The column Source Project contains a reference to the project for which the FP finding was produced; column Origin refers to the analyzer that produced the FP; columns CWE and ID show the reported CWE ID and the native vulnerability ID that is used by the analyzer to identify a vulnerability, respectively. The column False Positive Reason details why the reported finding can be considered as a false positive.

Source Project	Origin	CWE	ID	False Positive Reason
brakeman, flawfinder, php-security-audit, node-js-scan	gosec	78	G204	only variables to which constant values are assigned are used as parameters; variables are sanitized before used as parameters
eslint, kubesecc	gosec	703	G104	errors are handled implicitly by means of fallback default values; error cases are not relevant/can be ignored; returned error from an

anonymous function |
| kubesecc, mobsf | gosec | 22 | G304 | File content is checked to identify file type
returning a boolean value; filepath is already safely generated before being used |
| security-products | Gemnasium | CVE-2020-14040 | CVE-2020-14040 | Vulnerable code
found in imported libraries is dead-code and not called during run-time |
| security-products | Gemnasium | CVE-2020-29652 | CVE-2020-29652 | Vulnerable code
found in imported libraries is dead-code; i.e. not called at run-time |
| security-products | Gemnasium | GMS-2019-2 | GMS-2019-2 | YAML content parsed is not
user-generated |

SECTION: engineering/development/sec/secure/static-analysis/reaction_rotation.md

Reaction Rotation

Each milestone, two engineers in the team are assigned the role of Reaction Rotation, one as Primary and the other as Secondary. The assignments are in the rotation schedule.

The Secondary role is to step-in when the Primary is unavailable or over capacity. In such instances, the Secondary assumes the same responsibilities as the Primary, but otherwise they work on tasks planned for the milestone.

Responsibilities

The Reaction Rotation role has these responsibilities:

- Perform SAST Analyzer Vulnerability Management
- Respond to Requests for Help
- Answer Slack Questions
- Respond to GLAS Limitations Issues
- At the end of rotation update the membership for @gitlab-org/secure/static-analysis/reaction-rotation by removing the current engineers, and adding the next ones.

The Primary engineer can solicit help from the other engineers in the team. For example, when a task is not in their area of expertise, and they have already spent significant time (i.e. hours) without much progress; or when they're unable to keep-up with the volume of tasks.

Assistance should initially come from the Secondary engineer as to minimize disruption to milestone deliverables but all engineers should take into consideration that they may be pulled in to assist on a rotation task.

SAST Analyzer Vulnerability Management

The vulnerabilities for analyzers owned by Static Analysis need to be triaged and addressed.

1. Go to the list of vulnerabilities for SAST/IaC (sorted by SLO)
1. In decreasing order of severity, assign yourself to the issue.
1. Work on the issue:
 1. fix the bug; or

1. determine whether it needs to remain open, and handle the SLO exception process.

Requests for Help

Issues are created in the request-for-help project. During Reaction Rotation, the assigned engineer must review the open issues and engage with each issue.

If a request for help highlights a bug or feature request, create an issue in the public tracker, link to it in the request for help issue, and close the latter.

Slack Questions

Check the Static Analysis Slack Channel and respond to any questions asked or delegate/ping a person that may know the answer. As with Requests for Help, if the question concerns a bug or feature request, create an issue.

GLAS Limitations Issues

The Vulnerability Research team actively tests and enhances the GLAS rules. During this process, they identify various bugs and limitations of the engine. For each identified issue, they create detailed documentation. During milestone planning, five issues are selected and placed in an epic. The engineer assigned to Reaction Rotation must review and engage with each of these open issues.

Responsibilities of the rotation engineer:

1. Review and triage issues:
 - Assess whether the issue is a bug, engine limitation, or feature request.
2. For engine limitations or feature requests:
 - Document the limitation in the SAST assessment corpus.
 - Apply appropriate labels (e.g., `\~"feature::enhancement"` or `\~"feature::addition"`).
 - Ensure the issue is properly labeled and tagged for PM/EM attention.
3. For potential bugs:
 - Investigate to find the root cause (limit investigation to a few hours if GLAS is not your area of expertise).
 - If confirmed as a bug:
 - Apply the `\~"type::bug"` label.
 - Provide additional context in the issue to help with prioritization.
 - Tag the appropriate PM/EM for prioritization.
4. Update issue labels and information:
 - Adjust labels as necessary based on your investigation (e.g., changing from feature request to bug).
 - Add any relevant information or findings to the original issue.
5. Escalation and prioritization:
 - For critical bugs or issues requiring immediate attention, escalate to the PM/EM.
 - Provide enough context for the PM/EM to make informed decisions about prioritization.

Reaction Rotation Issue Tracking

To help track reaction rotation activities without unnecessary overhead, a simple issue

template is available for engineers to use. The template provides basic structure for documenting essential activities and includes helpful queries for monitoring key areas.

Engineers should use this template as a flexible tool to support their rotation work, adapting it as needed to fit their workflow while ensuring the core responsibilities outlined in this handbook are addressed.

SECTION: engineering/development/sec/secure/static-analysis/retrospectives_summary.md

Retrospectives

This page contains executive summaries of retrospectives done by the Secure::static analysis group. The purpose of these summaries is to externalize lessons learned during the retrospective process.

Access to internal retrospective documentation is available to the team and their immediate counterparts only.

17.0

This milestone focused on the deprecation and migration of functionality from various SAST analyzers to the semgrep based analyzer.

During this milestone the following concerns were raised in no particular order:

1. Lack of awareness of QA processes that failed during the final moments of the milestone caused confusion.
1. The release process for the sast-rules/semgrep pair is cumbersome and needs streamlining.
1. Implementation plans were not kept up to date during the milestone and caused unnecessary difficulty during the review process.

Specific Remediations raised during the discussion:

1. Undertake maintenance tasks to reduce complexity of semgrep & sast-rules release.
1. Apply strict review processes for implementation plans, and refer to the MVC principle more frequently

17.1

This milestone focused on SAST in the IDE and various maintenance tasks. During the retrospective discussion, the following points were raised:

1. The format of weekly team meetings is inefficient. The second iteration of the meeting often repeats discussions from the first iteration. To address this, the meeting format will now include "Carry-over" items that need further discussion in the next timezone. All other items

will be marked as read-only.

1. The policy for handling vulnerabilities is unclear. To improve this, we will create a runbook detailing the required steps.

17.2-17.3

During these milestones the team continued to work on SAST in the IDE. The retrospective focused on one main item:

1. We've discussed how blockers should be raised and how they should be handled. To enable a better reflection of status, we've changed the format of our standup to be more concise and will continue monitoring the situation.

SECTION: engineering/development/sec/secure/static-analysis/runbooks/_index.md

Overview

This page lists runbooks used by the Static Analysis team for monitoring, mitigating and responding to an incident.

Runbooks

How to monitor and respond to issues with SAST Automatic Vulnerability Resolution?
SAST analyzer deprecation and removal instructions
How to rollback an analyzer?

SECTION: engineering/development/sec/secure/static-analysis/runbooks/analyzer-deprecation-and-removal-instructions.md

Analyzer Conversion Lifecycle

Many of the SAST analyzers are in the process of being replaced by semgrep. This involves having semgrep takeover the functionality of the legacy analyzer.

The steps to achieve this are:

1. Migrate Rules to sast-rules
1. Audit Rules and review licensing
1. Deprecate and remove analyzers

This document is concerned with the Deprecate and remove analyzers step. All the deprecation steps must be completed before removal can commence.

Analyzer Deprecation

1. Deprecate the analyzer job in SAST.latest.gitlab-ci.yml

Submit an MR to update the SAST.latest.gitlab-ci.yml file by substituting the analyzer's entry with a placeholder. The placeholder message should provide a notice about when the analyzer's deprecation was announced and when it was removed. Be sure to include a hyperlink directing users to the corresponding deprecation issue for further details. Additionally, ensure the placeholder never runs by adding when: never to rules.

Always change the existing job to this new "placeholder" rather than deleting the job entirely. If we delete the entire job, this can break customer pipelines if they have provided custom variables or otherwise referenced the name of the deleted job.

yaml

script:

- echo "This job was deprecated in GitLab 14.8 and removed in GitLab 15.3"
- echo "For more information see <https://gitlab.com/gitlab-org/gitlab/-/issues/352554>"
- exit 1

rules:

- when: never

2. Deprecation Notification

We're required to publish notices in advance of potentially-breaking changes. See Deprecations, removals, and breaking changes for details on the required process.

An example of a previous SAST analyzer deprecation notice is the notice from 14.8.

The Product Manager and Engineering Manager for SAST are responsible for publishing this announcement.

Engineers implementing an analyzer removal are responsible for adhering to the stated scope of the change notice, and informing the PM/EM of any important changes to how customers will be affected by the change.

Analyzer Removal

1. Remove analyzer from documentation

All references to removed analyzers should be deleted from the SAST documentation. <https://docs.gitlab.com/ee/user/applicationsecurity/sast/>

Example of analyzer removal from documentation: <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/97451>

2. Add analyzer to ".End of supported analyzers" section of SAST documentation

All analyzers that are no longer supported should be listed in the end-of-supported-analyzers section of the SAST documentation.

3. Update the analyzer projects README.md

To further communicate that an analyzer is no longer supported, the following header should be added to its README.md.

text

Maintenance Notice:

This analyzer is currently in terminal maintenance mode. No new major versions will be released.

We've migrated this analyzer's scanning coverage to the GitLab SAST Semgrep-based analyzer.

4. Remove the analyzer job in SAST.gitlab-ci.yml

Submit an MR to update the SAST.gitlab-ci.yml file by substituting the analyzer's entry with a placeholder. The placeholder message should provide a notice about when the analyzer's deprecation was announced and when it was removed. Additionally, be sure to include a hyperlink directing users to the corresponding deprecation issue for further details.

You can use the previous update to SAST.latest.gitlab-ci.yml as a guide, but be sure to change the "removed in X" version number to match when the removal happened in the stable (SAST) template, not the latest (SAST.latest) template.

Example analyzer removal MR: <https://gitlab.com/gitlab-org/gitlab/-/mergerequests/97216>

5. Resolve all current vulnerabilities

Vulnerabilities created by an analyzer that has been removed should have their state set to resolved.

SECTION: engineering/development/sec/secure/static-analysis/runbooks/analyzer-rollback-production.md

Rolling back analyzer to previous minor or patch version

This runbook provides instructions for rolling back an analyzer to a previous version in case of a high severity incident in a faulty release. It is an immediate stop-gap measure to ensure a smooth user experience for our customer.

It is intended for engineers responsible for maintaining and troubleshooting issues in an analyzer.

Note: This runbook is relevant for analyzers that utilize the ci-templates by including either analyzer.yml, or docker.yml directly.

Rollback Procedure

First, identify the version of the analyzer where the bug appears. Test a prior version to ensure that the bug does not reproduce. Note any significant changes that were made between the two versions and record them in the bug issue. Consult with EM/PM if you are unsure that the

rollback is advisable.

Steps to rollback an analyzer

1. Navigate to the analyzer that needs to be rolled back
2. Go to the "Build" tag and select "Pipelines"
3. Click on the "Run pipeline" button
4. Choose the last known-good version (tag) of the analyzer
5. Run the pipeline
6. Wait for the pipeline to complete

Note: Once the pipeline finishes, it will automatically update the "latest" and major version image tag to the image built from the chosen tag.

Important Considerations

- Once a rollback has been made, no releases should be made until a bug fix has been merged to the default branch
- Only maintainers and owners are able to run pipelines on tags (they are protected)
- Verify the that known-good version before initiating the rollback
- Monitor the pipeline progress to confirm successful completion

Post-Rollback Actions

After successfully rolling back the analyzer:

1. Test that the bug in the rolled back analyzer image in production no longer replicates
2. Comment on the issue that a rollback has been made
3. Post an update in the group slack channel and tag all maintainers of the analyzer to ensure no MRs are merged until the bug is fixed
4. Notify the EM/PM that the production incident is mitigated
5. Plan for a new release with the bug fix

SECTION: engineering/development/sec/secure/static-analysis/runbooks/how-to-monitor-and-respond-to-issues-with-sast-automatic-vulnerability-resolution.md

When to use this runbook?

This runbook is intended to be used when there is a service degradation in relation to the SAST Automatic Vulnerability Resolution feature. Such degradation can be identified by monitoring the following:

Sidekiq Error Rate (in the Static Analysis group dashboard) with Vulnerabilities::MarkDroppedAsResolvedWorker selected.
Sidekiq execution Apdex and Error Ratio panels from the Static Analysis error budget.

SAST Automatic Vulnerability Resolution

The SAST Automatic Vulnerability Resolution feature is built to, as the name implies, automatically resolve vulnerabilities tied to SAST rules that have been disabled or removed.

The feature depends on a number of building blocks:

- Schema definition in security-report-schemas.
- SARIF module in analyzers/report package.
- Processing dropped identifier within Rails application.

Schema definition in security-report-schemas

Reports generated by security analyzer scans have their JSON schemas defined in security-report-schemas repository. Automatic vulnerability resolution depends on a certain schema field (i.e. primaryidentifiers) which is part of security-report-format, the latter being the parent schema for all other security reports' schema, including sast-report-format.

SARIF module in analyzers/report package

The primaryidentifiers field contains an exhaustive list of all identifiers for which the analyzer scan (as opposed to identifiers detected), so a report may have zero vulnerabilities but scan.primaryidentifiers contain a full list. The list is generated while transforming a SARIF file into a SAST security report in the sarif.go module under analyzers/report package.

Dropped identifier processing within Rails application

While ingesting a security report within the gitlab-org/gitlab application, the IngestReportsService iterates through scan primary identifiers and executes ScheduleMarkDroppedAsResolvedService for each scan type, which in turn schedules MarkDroppedAsResolvedWorker. The worker loops through all vulnerabilities with identifiers matching the disabled or dropped identifiers (i.e no longer present in latest scan).

Below is a diagram showing the complete flow of automatic vulnerability resolution feature.

mermaid

flowchart TB

code --> analyzerpipeline

```
subgraph analyzerpipeline["analyzer pipeline"]
  direction LR
  analyzer["semgrep analyzer"] --> reporta["noisy-rule-123 dropped"]
  reporta --> reportb["scan.identifiers populated"]
  reportb --> reportc["gl-sast-report.json"]
end
```

analyzerpipeline --> railsapplication

```
subgraph railsapplication["rails application"]
  ingest["IngestReportsService"] --> schedule["ScheduleMarkDroppedAsResolved"]
  schedule --> worker["MarkDroppedAsResolvedWorker"]
end
```

Monitoring

To monitor automatic vulnerability resolution, there are two primary sources of information: sentry.io which lists any errors occurring in MarkDroppedAsResolvedWorker class for the last 24 hours, and SAST Engineering dashboard on Kibana, which includes a number of panels monitoring certain works and showing the volume of uploaded reports. Please see below for a list of panels of interest and a brief description of each.

SAST Report Uploads

Displays the 90th percentile of file size of security reports uploaded, per 30 minutes. This is useful to see how big (or small) security reports that have been uploaded over a certain amount of time.

SAST Failing Workers Distribution

Shows the distribution of SAST-related sidekiq workers failing over a period of time.

Vulnerabilities::MarkDroppedAsResolvedWorker Execution Time

Displays the 75th and 95th percentiles of the worker's execution time.

Vulnerabilities::MarkDroppedAsResolvedWorker Job Status

Shows the count of job executions, split by job status, per hour. This is useful to gauge the amount of failing, deduplicated, or successful executions over a certain amount of time.

Top Projects for MarkDroppedAsResolvedWorker Executions

Shows the top projects listed by the count of their worker executions. This can be useful to see if a certain customer is experiencing an issue.

Logs

Additionally, you may want to check the following two saved searches in production logs:

Vulnerabilities::MarkDroppedAsResolvedWorker · Total Executions.

Vulnerabilities::MarkDroppedAsResolvedWorker · Executions with DB Writes.

What to do if something goes wrong?

1. Start by looking at the monitoring section above. Check if MarkDroppedAsResolvedWorker has any failures.
1. Look at the logs, and see if the issue is possibly due to a query timing out while executing a database write operation (e.g. trying to resolve a huge number of findings).
1. Consider turning automatic vulnerability resolution off.

Possible Checks

If there's an increase in error rates in relation to automatic vulnerability resolution, there's a possibility it could be related to this timeout issue when a very high number of vulnerability findings are being resolved.

How to turn automatic vulnerability resolution off?

The presence of primaryidentifiers is required for report ingestion and automatic vulnerability resolution. If automatic vulnerability resolution is not working as expected, consider stopping automatic resolution by ensuring scans do not have primaryidentifiers included in the generated reports. To do so, consider one of the following options:

1. Update sarif.go module to revert the change introduced in this merge request.
2. Update ScheduleMarkDroppedAsResolvedServiceDroppedIdentifiers method to return early regardless of the existence of primaryidentifiers.

SECTION: engineering/development/sec/secure/tech-docs/_index.md

Architecture

- Overview
- Severity Levels
- Feedback(Dismiss, create an issue or a Merge Request)

Overview

The architecture supporting the Secure features is split into two main parts.

```
mermaid
flowchart LR
    subgraph G1[Scanning]
        Scanner
        Analyzer
        CI[CI Jobs]
    end
    subgraph G2[Processing, visualization, and management]
        Parsers
        Database
        Views
        Interactions
    end
    G1 --Report Artifact--> G2
```

Scanning

The scanning part is responsible for finding vulnerabilities in given resources and exporting results.

The scans are executed in CI jobs via several small projects called Analyzers which can be

found in our Analyzers sub-group.

The Analyzers are small wrappers around in-house or external security tools called Scanners to integrate them into GitLab.

The Analyzers are mainly written in Go and rely on our Common Go library.

Some 3rd party integrators also make additional Scanners available by following our integration documentation, which leverages the same architecture.

The results of the scans are exported as JSON reports that must follow Secure Report Format and are uploaded as CI Job Report Artifacts to make them available for processing after the pipelines completes.

This part is mainly covered by the following groups:

- Composition Analysis
- Dynamic Analysis
- Fuzz Testing
- Static Analysis
- Vulnerability Research

Processing, visualization, and management

Once the data is available as Report Artifact, it can be processed by the GitLab Rails application to enable our security features:

- Security Dashboards, Merge Request widget, Pipeline view, etc.
- Interactions with vulnerabilities
- Approval rules
- etc.

Depending on the context, the security reports can be stored in the database or stay as Report Artifacts for on-demand access.

This part is mainly covered by the Threat Insights group.

Though, the boundaries can sometimes be a bit blurry so we're trying to delineate this as clearly as possible.

ClickHouse Datastore

Key workloads across the Secure features rely on high rates of writes and aggregated analysis across historic data. These types of OLAP scenarios are not well-suited to transactional datastores like PostgreSQL and can benefit from batch-based inserts, high-read ratios, and wide tables.

In these cases, the introduction of ClickHouse to the GitLab technology stack provides an important opportunity to scale Secure features for the future.

ClickHouse as a datastore has the potential to power several key workflows within the section including:

Security Dashboards

Security dashboards provide historical aggregate data for tracking active vulnerabilities across projects and namespaces. These requests are analytical aggregation queries of read-only data for which ClickHouse is heavily optimized.

Beyond improving the performance of the existing aggregations, use of an OLAP datastore provides more open-ended options in allowing on-demand aggregation by additional fields; i.e. report types and classifications alongside severity.

Vulnerability Lists

Vulnerability lists provide tabular data and interactivity for reviewing, assessing, and triaging vulnerabilities within projects and namespaces. These requests are high-read, wide-column and (often) filtered. With a shift towards query-based view aggregation, columnar stores provide significant advantages in fetching limited columns for a given table rather than needing full record access.

In addition, with ongoing architecture aimed at reducing persistence to user-interaction such as the work to Create Vulnerabilities on-the-fly there is a lot of potential for performance improvements in shifting vulnerability finding storage to ClickHouse over PostgreSQL.

Researches

- Data model for dependencies information

Brown bag sessions

Secure team members also share knowledge through brown bag sessions on various topics.

SECTION: engineering/development/sec/secure/tech-docs/data-model-for-dependencies-information.md

Overview

This document explores additions to the Security reports and to the database schemas that would enable new features for Dependency Scanning and License Scanning.

The solutions being considered are:

- adding tables to the PostgreSQL primary database (schema change)
- creating a new PostgreSQL database (new schema)
- storing new documents in Object Storage
- indexing data in Elasticsearch
- caching data in Redis

Most sections of this document are organized in three parts:

1. feature: What do users need?

1. technical discussion: How can we implement the feature? What are the technical constraints, and how can we satisfy them?

1. proposal: What is the corresponding DB schema or document schema?

This document contains a simplified proposal that should be used when implementing new features.

As such, it might skip implementation details, like the exact Foreign Key constraints we would add to the relational DB.

Dependency search

Users want to search project dependencies:

- for a specific package name; they might also specify a package type and a version number to refine the search
- for packages that use a given license
- for packages that are affected by a given vulnerability

The scope of the search can either be:

- a single project
- a project group/namespace
- a GitLab instance

The search result is a list of packages that match the query along with the projects that depend on these.

In a first version the search is limited to the default branch.

Later on, it could be extended to all protected branches or to all project releases.

Search for a package

Users want to know what package versions their projects use.

They might search project dependencies for:

- a package name
- a package name and a version number

When searching project dependencies, users might not know the non-canonical package names. For instance, they might search for django, not realizing that Django is the canonical name for that Python package.

Also, django might be the dependency name used in their project, in the dependency file.

We need a DB table to efficiently query project dependencies by package name, type, and version.

We could have a projectdependencies table with:

- smallint column for the package type (integer values are interpreted by the Rails backend)
- varchar column for the package name

- varchar column for the package version
- composite index that combines all these columns, for performance

However, package names and versions would be repeated across project dependencies:

- Some packages are popular; many projects depend on them, and their names are repeated.
- This also applies to specific versions, and both the names and versions are repeated.
- In the future, the search won't be limited to the default branch, and branches of the same project are likely to repeat names and versions.

In order to save storage, we keep track of packages used as project dependencies in two separate tables:

- packages tracks packages by type and canonical name
- packageversions tracks versions by packageid and version

sql

```
CREATE TABLE packages AS (  
  id integer NOT NULL,  
  type smallint NOT NULL,  
  name varchar NOT NULL  
);
```

```
CREATE TABLE packageversions AS (  
  id integer NOT NULL,  
  packageid integer NOT NULL,  
  version varchar NOT NULL  
);
```

To reduce storage, these tables can be cleaned up when packages or specific versions are no longer used as project dependencies.

This can be done when updating project dependencies, or in a periodically executing batch job.

Since packages and packageversions records are coming from the scanning jobs and these can't be trusted,

the default scopes for the corresponding ActiveRecord models shouldn't return any objects.

In the future, these scopes could return all the packages and versions that match what's listed on trusted public package registries.

Since the packages table doesn't track the package source,

the same record might refer to a public package (fetched from the main registry) or to a private package.

This collision is not a problem since the goal is to enable efficient queries with a limited impact on storage,
and not to accurately track packages.

The canonical names stored in the DB might differ from the package names used in the dependency file.

The Rails backend normalizes the names before storing them in the packages table.

When processing a search query, the backend normalizes the given package names so that they can be compared to the ones stored in the DB.
It might also make the search case-insensitive for convenience.
If needed, the exact package names used in the dependency files can be fetched from the Dependency Scanning report and presented in the search result.

To support multi-project repositories, we need to track the paths to the dependency files where project dependencies were found, and report these files in the search result. This supports the case where the same package manager is used in different directories of the same GitLab project, like client/Gemfile and server/Gemfile. Also, the file paths are likely to be repeated across projects, and they should be stored in a separate DB table to reduce storage.

To avoid repeating the file paths across dependencies of the same project and across projects, we track project dependencies using 2 tables:

- dependencyfiles tra